Coupon Usage Prediction Model

Machine Learning for Customer Transaction Analysis

Name Goes Here

University Goes Here

February 28, 2026

Problem Statement

Research Question: Do transaction patterns and discount behaviors predict coupon usage?
Goal: Improve coupon targeting and reduce marketing waste
Target Variable:

- `coupon_used` (1 = used, 0 = not used) - derived from coupon usage records

Problem Type: Binary classification

- Features: transaction amounts, customer behavior, time patterns
- Response: whether a customer will use a coupon in a transaction

Data Description

Data Sources:

Dataset	Records	Description
Wallet Transactions	490,942	Customer purchases
Coupon Usage	75,676	Coupons actually used
Coupon Distribution	211,712	Coupons sent to customers

Key Patterns to Explore:

- Relationship between transaction amount and coupon usage
- Impact of discount depth on redemption rates
- Time-based patterns (hour, day of week)
- Customer behavior history as predictor

Variable Types

Numeric (8):

- tran_amt, discounts_amt, point_amt
- coupon_used_count, total_coupon_used_amt
- coupon_send_count, total_coupon_send_amt
- benefit_ratio, discount_ratio, savings_pct

Categorical (5):

- station_code
- attributionorgcode
- transactionorgcode
- hour, day_of_week
- is_weekend, is_morning

Target Variable: coupon_used (Binary: 1 = Used, 0 = Not Used)

Sample Data: Raw Input (Pandas)

Code

```
df = pd.read_csv('cust_wallet_detail_train.csv')
df = df.merge(coupon_used_agg, on='membercode', how='left')
df = df.merge(coupon_send_agg, on='membercode', how='left')
```

pd.head(5) - Raw Data (Merged)

```
  membercode  station_code  tran_amt  receivable_amt  discounts_amt  coupon_amt  attributionorgcode
      coupon_used_count  total_coupon_used_amt
1.045000e+12      33300724     220.0           220.0            0.0         0.0                45.0
                   4482                 3803400
1.045010e+12      33303134     100.0           100.0            0.0         8.0                45.0
                   6186                 5223300
1.045010e+12      33300659     200.0           200.0            0.0         0.0                45.0
                   6186                 5223300
1.045010e+12      33300144     200.0           200.0            0.0         0.0                45.0
                   6186                 5223300
```

Sample Data: Processed Features (Pandas)

pd.head(5) - Processed Data

```
station_prefix_encoded  attributionorgcode  hour  tran_amt  receivable_amt  tran_to_receivable_ratio
     day_of_week  total_coupon_send_amt  coupon_used_count  transactionorgcode  total_coupon_used_amt
     coupon_send_count  savings_pct  benefit_ratio  discount_ratio  coupon_used
                    0                45.0    22     220.0           220.0                  0.995475
                        3                9869780                4482                  45                  3803400
                                        9934          0.0            0.0             0.0             0
                    0                45.0    17     100.0           100.0                  0.990099
                        3                14753390               6186                  45                  5223300
                                       14809          0.0            0.0             0.0             1
                    0                45.0    14     200.0           200.0                  0.995025
                        3                14753390               6186                  45                  5223300
                                       14809          0.0            0.0             0.0             0
                    0                45.0     8     200.0           200.0                  0.995025
                        3                14753390               6186                  45                  5223300
                                       14809          0.0            0.0             0.0             0
```

Key Libraries

- **pandas**: Data manipulation and analysis
- **numpy**: Numerical computations
- **sklearn**: Machine learning models and evaluation
- **tensorflow**: Neural network implementation
- **matplotlib**, **seaborn**: Data visualization

Key Functions

Data Processing:

- `pd.read_csv()`: Load datasets
- `df.merge()`: Join datasets
- `df.drop()`: Remove columns
- `df.quantile()`: Outlier removal

Modeling:

- `GradientBoostingClassifier()`: Build model
- `cross_val_score()`: Validate
- `feature_importances_`: Get importance

Key Functions (sklearn)

Data Split:

- `train_test_split()`: Split data (80/20)
- `StratifiedKFold()`: 3-fold CV
- `StandardScaler()`: Feature normalization

Dimensionality Reduction:

- `PCA()`: Principal component analysis
- `TSNE()`: t-SNE embedding
- `KernelDensity()`: Density estimation

Model Evaluation:

- `accuracy_score()`: Accuracy metric
- `roc_auc_score()`: AUC metric
- `classification_report()`: Precision/Recall/F1
- `confusion_matrix()`: Confusion matrix
- `cross_val_score()`: Validation

Key Functions (matplotlib, seaborn, tensorflow)

Model Training:

- `RandomForestClassifier()`: Random Forest
- `GradientBoostingClassifier()`: Gradient Boosting
- `SVC()`: Support Vector Machine
- `LogisticRegression()`: Logistic Regression

Visualization (matplotlib, seaborn):

- `plt.savefig()`: Save plots
- `sns.heatmap()`: Correlation heatmap
- `ConfusionMatrixDisplay()`: Confusion matrices

Neural Network (tensorflow):

- `keras.Sequential()`: Build model
- `layers.Dense()`: Dense layers
- `layers.BatchNormalization()`: Batch norm
- `layers.Dropout()`: Dropout
- `model.compile()`: Compile model
- `model.fit()`: Train model
- `model.predict()`: Predictions

Data Preprocessing I

1. **Feature Selection (Removed IDs/Hashes):**
   - `membercode`, `order_no`, `external_order_no`
   - `coupon_code`, `user_id`
2. **Outlier Removal (IQR Method):**
   - Method: Removed values outside 5th-95th percentile (IQR-based)
   - Applied to: `tran_amt`, `discounts_amt`, `point_amt`
   - Why: Extreme values distort model training and visualization
   - Records reduced: 490,942 $\rightarrow$ 421,590 (14% removed)
3. **Colinearity Removal:**
   - `receivable_amt` (99% correlated with `tran_amt`)
   - `net_amount`, `total_benefit` (redundant)

Features I

Remaining Features (15):

- Transaction: `tran_amt`, `discounts_amt`, `point_amt`
- Customer/Store: `station_prefix_encoded`, `attributionorgcode`, `transactionorgcode`
- Time-based: `hour`, `day_of_week`, `is_weekend`, `is_morning`, etc.
- Aggregated: `coupon_used_count`, `total_coupon_used_amt`
- `coupon_send_count`, `total_coupon_send_amt`

Engineered Features (5):

- `benefit_ratio`, `discount_ratio`
- `savings_pct`, `point_to_discount_ratio`
- `tran_to_receivable_ratio`

Feature Selection

Removed Irrelevant Features (IDs and Hashes):

- `membercode` - Customer ID (hash)
- `order_no` - Transaction ID (hash, no date pattern)
- `external_order_no` - External ID (hash)
- `coupon_code` - Coupon ID (hash)
- `user_id` - User ID (hash)

Impact: Removing `membercode` improved accuracy from 64% to 66%

Feature Engineering: Station Code

Problem: High-cardinality numeric feature

- `station_code`: 18,806 unique values (treated as numeric)
- PCA showed it dominated variance (54% on PC1)
- 23.5% feature importance - but likely overfitting

Discovery: Extract station prefix

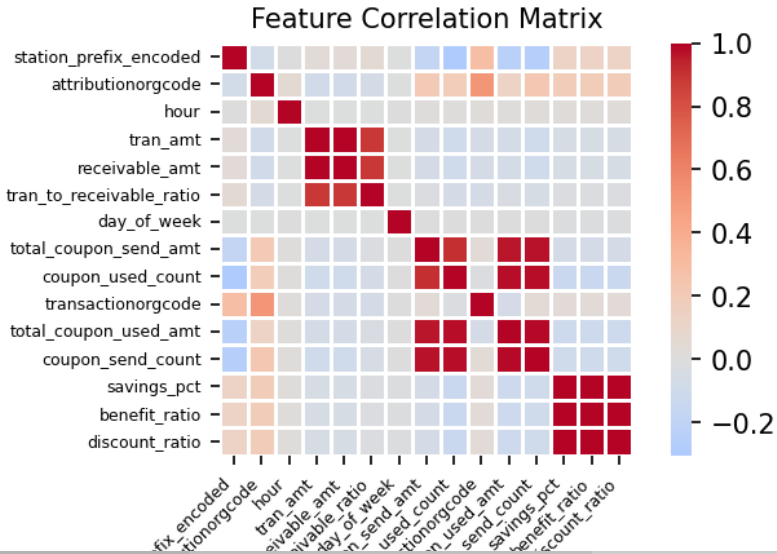
- First 3 digits = store channel/region
- Only 19 unique values
- Different prefixes show 22-54% coupon usage rates

Experiment Results (Random Forest):

- With station_code: 74.2% accuracy
- Without station_code: 74.4% accuracy
- With station_prefix: **74.6% accuracy**

Result: station_prefix has lower importance (4.7%) but improves generalization.

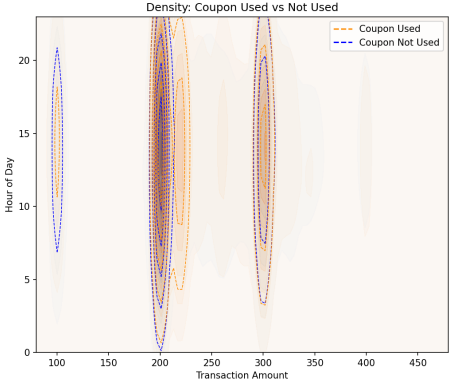
Correlation Matrix Heatmap

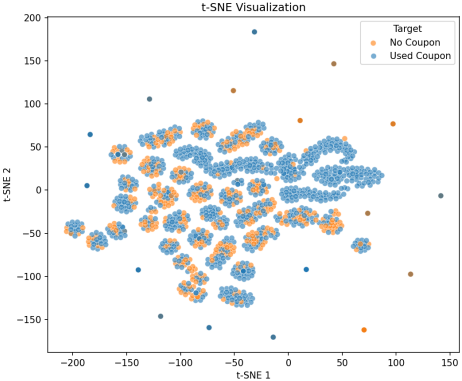


Density Plot Analysis

Dimensionality Reduction:

- Kernel density estimation (KDE) shows probability distribution
- Overlay of two classes on same axes for comparison
- X-axis: Transaction amount or Hour, Y-axis: Density





t-SNE Visualization

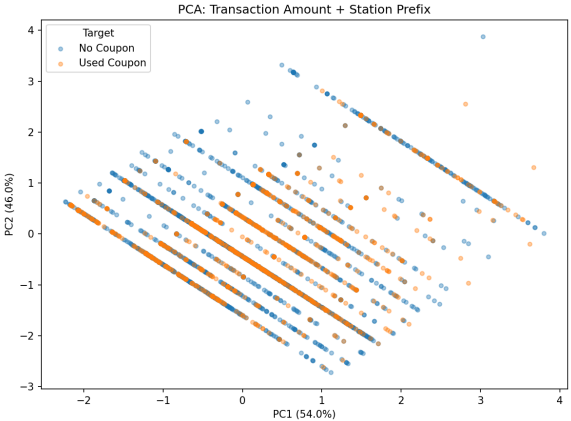
t-SNE (t-Distributed Stochastic Neighbor Embedding):

- Non-linear dimensionality reduction
- Preserves local structure (similar points stay close)
- Perplexity=30 used
- Reveals class clusters in 2D space

Key Observations from Density Plot and t-SNE:

- Coupon users (orange) tend to have higher transaction amounts
- Non-users (blue) concentrate in lower amounts
- Both methods show significant class overlap
- No clear linear boundary between classes
- Suggests other features needed for better separation

PCA Visualization I



Key insight: station_code prefix matters

- Raw station_code: 18K unique (acts like ID)
- Prefix (first 3 digits): 19 unique groups
- Prefix shows clear coupon usage patterns:
  - 331, 310, 328: 52% usage
  - 334, 335: 22% usage
- Likely represents store channel/region

PCA Visualization II

What the PCA shows:

- Vertical bands = different station prefixes
- PC1 (54%) captures station prefix
- PC2 (46%) captures transaction amount
- Within each band: classes overlap significantly
- Station prefix is the key differentiator

Implication for model:

- Feature engineering: extract station prefix
- Station/channel is more predictive than transaction amount

Model Architecture

Random Forest Parameters:

- n_estimators=100
- max_depth=20
- min_samples_split=15
- min_samples_leaf=3
- max_features="sqrt"

Gradient Boosting Parameters:

- max_iter=100
- max_depth=10
- learning_rate=0.1

SVM Parameters:

- kernel="rbf"
- C=1.0
- gamma="scale"

Logistic Regression Parameters:

- max_iter=1000
- solver="lbfgs"
- class_weight="balanced"

Data Split: Train/Test: 80/20, Stratified

Neural Network Architecture

Neural Network Details:

- Input Layer: 15 features
- Hidden Layer 1: 256 neurons (ReLU)
- BatchNormalization + Dropout(0.3)
- Hidden Layer 2: 128 neurons (ReLU)
- BatchNormalization + Dropout(0.3)
- Hidden Layer 3: 64 neurons (ReLU)
- BatchNormalization + Dropout(0.3)
- Output Layer: 2 neurons (Softmax)

Training:

- Optimizer: Adam (lr=0.001)
- Loss: Categorical Crossentropy
- Validation Split: 10%
- Batch Size: 256
- Epochs: 100

Neural Network: Class Weights

Impact of Class Weights on Neural Network:

Naive (Unbalanced) Weights: {0:1, 1:2}

- Accuracy: 62.92%
- Recall: **92.06%**
- Precision: 49.59%
- Result: Predicts almost everything as positive

Balanced Weights: {0:1, 1:1}

- Accuracy: **71.85%**
- Recall: 58.11%
- Precision: **62.29%**
- Result: Better balance between precision and recall

Key insight: Naive weights over-predict positives, achieving high recall but low precision. Balanced weights improve overall accuracy and create a more useful model.

Model Performance (20,000 sample)

Metric	Random Forest	SVM	Gradient Boost	Logistic Reg	Neural Net
Accuracy	**74.60%**	70.80%	73.68%	60.48%	72.22%
AUC	**0.825**	0.761	0.818	0.676	0.799
Precision	65.40%	61.23%	63.60%	47.20%	62.70%
Recall	64.68%	54.69%	65.30%	69.27%	59.14%
F1 Score	**65.04%**	57.77%	64.44%	56.14%	60.87%

Random Forest achieves best accuracy with station_prefix feature engineering (+0.4% vs old station_code).

Sample Size Comparison

Model	20,000 Accuracy	421,590 Accuracy	Improvement
Random Forest	74.22%	**75.60%**	+1.38%
Gradient Boosting	73.62%	**74.13%**	+0.51%

Larger sample size improves model accuracy, with Random Forest showing greater benefit from more data.

Hyperparameter Tuning (Grid Search on Sample Size of 20,000)

Grid Search Results vs Default Parameters:

Random Forest:

- Best params: max_depth=10, min_samples_leaf=1, n_estimators=200
- Grid Search: 73.80% accuracy
- Default: 74.60% accuracy
- Conclusion: Default params slightly better

Gradient Boosting:

- Best params: learning_rate=0.1, max_depth=15
- Grid Search: 73.68% accuracy
- Default: 73.68% accuracy
- Conclusion: Similar performance

Hyperparameter Tuning Results (cont.)

SVM:

- Best params: C=10, gamma='auto'
- Grid Search: 71.63% accuracy
- Default: 70.80% accuracy
- Conclusion: +0.8% improvement

Summary:

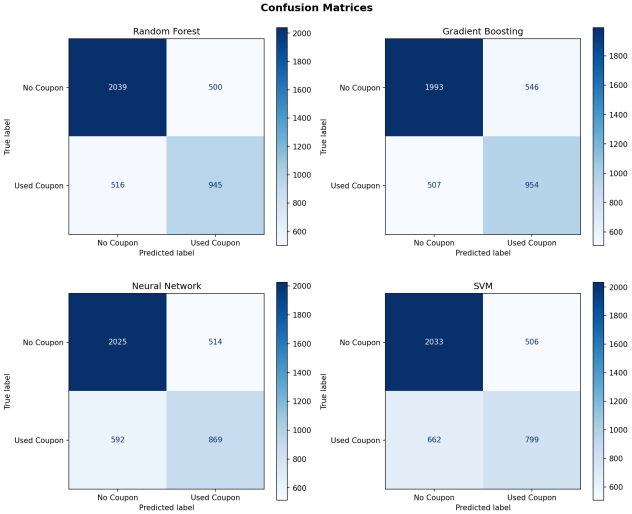
- Original parameters were well-tuned
- Grid search did not significantly improve results
- Random Forest remains best model
- Default RF params (74.6%) > Grid Search (73.8%)

Grid Search Results Summary

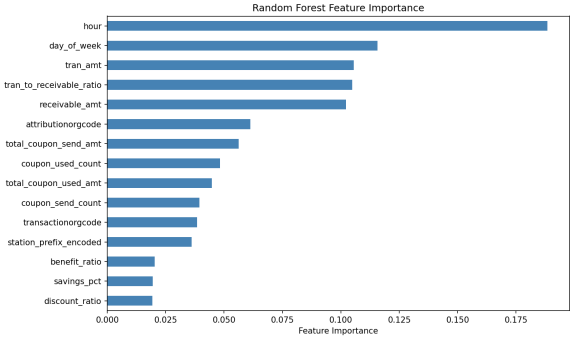
Model	ROC	AUC
Random Forest	**73.80%**	**82.02%**
Gradient Boosting	73.62%	81.72%
Support Vecotr Machines	71.63%	77.40 %

- Not a marginal improvement on initial model performance
- With more time and resources, hope to expand the grid

Confusion Matrices



Feature Importance from Random Forests



Top Features:

1. `hour` (15.4%)
2. `tran_to_receivable_ratio` (10.5%)
3. `tran_amt` (10.5%)
4. `receivable_amt` (10.2%)
5. `day_of_week` (9.3%)
6. `attributionorgcode` (7.5%)
7. `total_coupon_send_amt` (5.4%)
8. `total_coupon_used_amt` (4.9%)
9. `coupon_used_count` (4.8%)
10. `coupon_send_count` (4.7%)
11. `station_prefix_encoded` (4.7%)

How Feature Importance was Derived:

- Using Random Forest classifier's built-in feature importance
- Measures mean decrease in impurity (Gini importance)
- Averaged across all 100 decision trees
- Higher value = more predictive power

Key Insight: Time features (hour, day) and transaction amounts are most predictive. Station prefix has lower feature importance (4.7%) but improves model generalization (74.6% accuracy vs 74.2%).

Conclusions

- Random Forest achieves **74.60% accuracy** with **0.825 AUC** - best model
- Gradient Boosting: 73.68% accuracy, 0.818 AUC
- Neural Network: 72.22% accuracy with balanced class weights
- Feature selection: reduced from 23 to 15 features without accuracy loss
- Replaced station_code with station_prefix for better generalization
- Top features: hour (15.4%), tran_to_receivable_ratio (10.5%)
- 3-Fold Cross Validation confirms model stability

Future Work

Potential Improvements:

- **Hyperparameter Tuning**: Grid search or random search for optimal parameters. Explore other kernels, C and γ sizes for SVM.
- **Advanced Models**: XGBoost, LightGBM for potentially better performance
- **Feature Engineering**: More customer behavior features, time-series patterns
- **Class Imbalance**: Explore SMOTE, undersampling techniques
- **Deployment**: Build API for real-time coupon prediction

Repository

GitHub Repository:
https://github.com/alexanderneva/statscoupons

All code, data, and presentation materials are available in this repository.